

Universidad de Monterrey

Facultad de Ingeniería



Proyecto de introducción a la ingeniería

Título:

A2.1 Equipo 7

Profesor: Antonio Martínez Torteya

Alumnos:

Ines Castillo Enríquez, 692934

Julián Sabino Duque Saldívar, 691784

Karol Alessandra Sandoval Lara, 650421

Emiliano García Vidales, 692086

“Doy mi palabra que he realizado esta actividad con integridad académica”

Introducción

Para esta actividad, nuestro equipo moduló la velocidad de cuatro motores mediante botones, esto para que el vehículo que queremos hacer pueda cambiar de dirección y, dependiendo de a qué dirección vaya, las llantas irán a sentido horario o antihorario.

Metodología

I. Conexiones del sistema

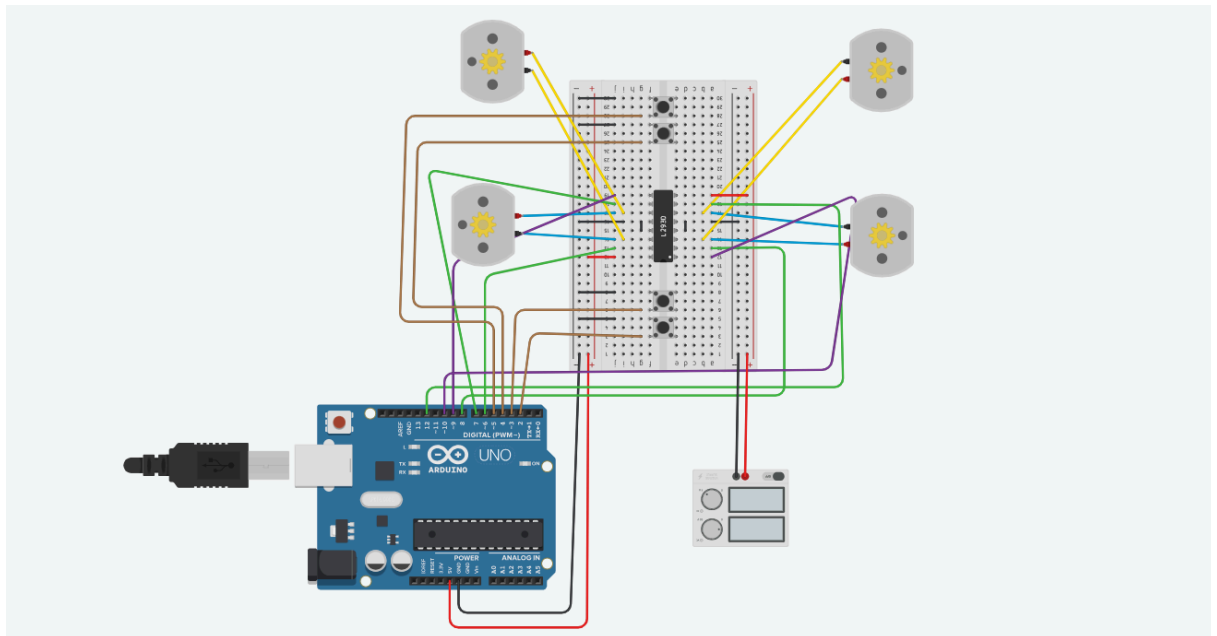


Figura 1: Componentes del sistema en TinkerCAD

1. Descripción y Justificación de Componentes

- **Arduino Uno:** Es el que proporciona los puertos físicos de entrada para leer variaciones de voltaje y los puertos de salida capaces de emitir señales de control.
- **4 Motores CC:** Transforma la energía eléctrica del circuito de 9V en energía mecánica mediante movimiento rotativo continuo.
- **Motor Driver L2930:** Funciona para poder controlar los motores al mismo tiempo.

- **Resistor:** Se usa para restringir el flujo de electrones que evita que ocurra una sobrecorriente que destruya tanto los puertos de entrada como el propio transistor.
- **Botones:** Sirven para poder pasar o no pasar corriente dependiendo de si está apretado o no.
- **Protoboard y cableado:** Base de ensamble temporal y medios conductores que establecen la conectividad del circuito.

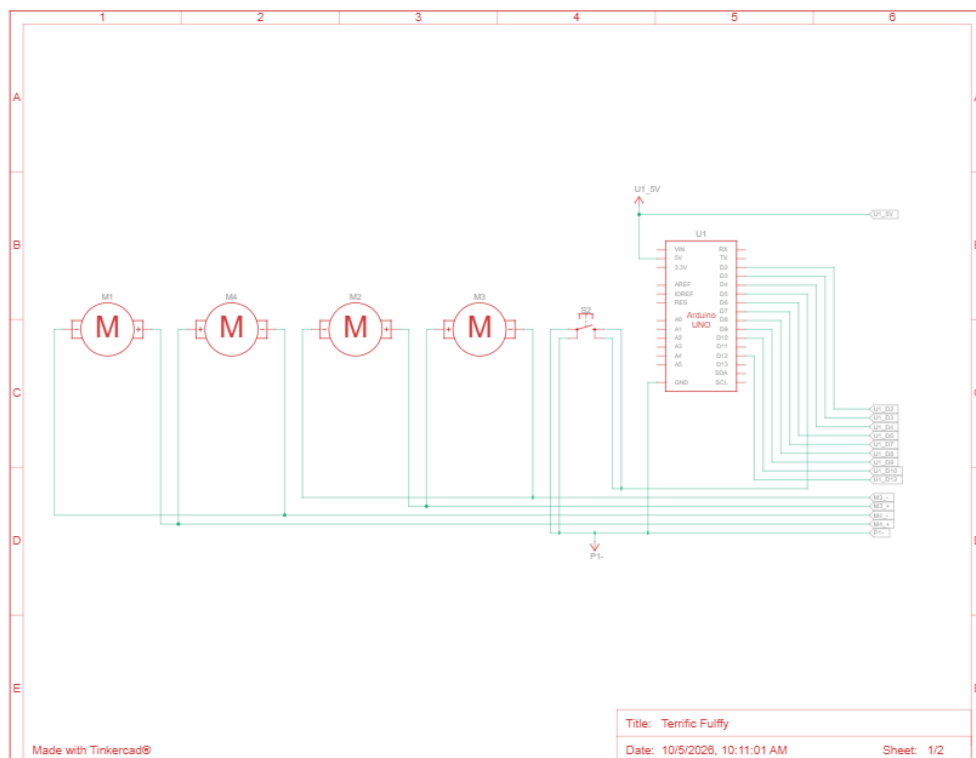


Figura 2: Diagrama de las conexiones del sistema generado por TinkerCAD

2. Ensamblaje y Justificación de Nodos

- Se separó la alimentación lógica (5V) de los motores de la energía externa, pero se unieron sus cables de tierra (cable negro entre rieles). Esto permite crear una referencia común de 0V, lo cual ayuda al circuito de potencia a entender correctamente las señales del microcontrolador.
- El puente H (L293D) recibe instrucciones desde los pines PWM del Arduino, lo que permite controlar tanto la dirección como la velocidad de los motores.

- Los cuatro motores se conectaron en pares paralelos a las salidas del integrado, formando una configuración típica de tracción diferencial (como la de un tanque).
- Se instalaron cuatro botones en el protoboard que controlarán las acciones de cada motor.
- Esta configuración Pull-down asegura un estado bajo (LOW) por defecto, lo que elimina el ruido eléctrico y envía una señal limpia de 5V (HIGH) al presionar, activando así los comandos.

II. Código

```

1  // ===== BOTONES =====
2  const int boton1 = 2;  // Adelante
3  const int boton2 = 3;  // Atrás
4  const int boton3 = 4;  // Derecha
5  const int boton4 = 5;  // Izquierda
6
7  // ===== DRIVER LADO IZQUIERDO =====
8  const int IN1 = 6;
9  const int IN2 = 7;
10 const int ENA = 9;
11
12 // ===== DRIVER LADO DERECHO =====
13 const int IN3 = 8;
14 const int IN4 = 12;
15 const int ENB = 10;
16
17 // Velocidad de los motores
18 const int velocidad = 200;

```

Figura 3: Declaración de variables

Como se muestra en la figura 3, las primeras líneas del código corresponden a la declaración de múltiples variables de tipo entero. Las primeras cuatro corresponden a los botones y las siguientes 6 a los motores. Asimismo, a todas se les asignó el valor del pin al que su respectivo componente fue conectado. Finalmente, se creó una variable para la velocidad, a la cual se le dio un valor inicial de 200.

```

20 void setup() {
21
22     // Botones
23     pinMode(boton1, INPUT_PULLUP);
24     pinMode(boton2, INPUT_PULLUP);
25     pinMode(boton3, INPUT_PULLUP);
26     pinMode(boton4, INPUT_PULLUP);
27
28     // Motor izquierdo
29     pinMode(IN1, OUTPUT);
30     pinMode(IN2, OUTPUT);
31     pinMode(ENA, OUTPUT);
32
33     // Motor derecho
34     pinMode(IN3, OUTPUT);
35     pinMode(IN4, OUTPUT);
36     pinMode(ENB, OUTPUT);
37
38     detener();
39 }
40
41 void loop() {
42
43     if (digitalRead(boton1) == LOW) {
44         adelante();
45     }
46
47     else if (digitalRead(boton2) == LOW) {
48         atras();
49     }
50
51     else if (digitalRead(boton3) == LOW) {
52         derecha();
53     }

```

Figura 4: Función “void setup” junto con parte de la función “void loop()”

Posteriormente, dentro de la función “void setup()” (que se ejecuta una sola vez) del código, se configuraron los pines de los botones como entradas, junto con la función “INPUT_PULLUP”, para que se ejecute una resistencia interna, se configuraron como entrada ya que de ellos el Arduino recibirá información. Además, los pines de los motores se configuraron como salidas, ya que por ellos el Arduino enviará señales de control. ENA y ENB son los pines de habilitación para controlar la velocidad mediante modulación por ancho de pulsos (PWM) para el lado izquierdo y derecho, respectivamente. Por último, se usó la función “detener ()” al final para establecer que los motores estarán completamente apagados cuando se encienda el circuito.

1

```

54
55 else if (digitalRead(boton4) == LOW) {
56     izquierda();
57 }
58
59 else {
60     detener();
61 }
62 }
63
64 // ===== ADELANTE =====
65 void adelante() {
66
67     // Lado izquierdo
68     digitalWrite(IN1, HIGH);
69     digitalWrite(IN2, LOW);
70
71     // Lado derecho
72     digitalWrite(IN3, HIGH);
73     digitalWrite(IN4, LOW);
74
75     analogWrite(ENA, velocidad);
76     analogWrite(ENB, velocidad);
77 }
78
79 // ===== ATRÁS =====
80 void atras() {
81
82     // Lado izquierdo
83     digitalWrite(IN1, LOW);
84     digitalWrite(IN2, HIGH);
85

```

2

```

86 // Lado derecho
87 digitalWrite(IN3, LOW);
88 digitalWrite(IN4, HIGH);
89
90 analogWrite(ENA, velocidad);
91 analogWrite(ENB, velocidad);
92 }
93
94 // ===== GIRAR DERECHA =====
95 void derecha() {
96
97     // Izquierdos adelante
98     digitalWrite(IN1, HIGH);
99     digitalWrite(IN2, LOW);
100
101     // Derechos atrás
102     digitalWrite(IN3, LOW);
103     digitalWrite(IN4, HIGH);
104
105     analogWrite(ENA, velocidad);
106     analogWrite(ENB, velocidad);
107 }
108
109 // ===== GIRAR IZQUIERDA =====
110 void izquierda() {
111
112     // Izquierdos atrás
113     digitalWrite(IN1, LOW);
114     digitalWrite(IN2, HIGH);
115
116     // Derechos adelante
117     digitalWrite(IN3, HIGH);
118     digitalWrite(IN4, LOW);
119

```

3

```

120     analogWrite(ENA, velocidad);
121     analogWrite(ENB, velocidad);
122 }
123
124 // ===== DETENER =====
125 void detener() {
126
127     analogWrite(ENA, 0);
128     analogWrite(ENB, 0);
129
130     digitalWrite(IN1, LOW);
131     digitalWrite(IN2, LOW);
132     digitalWrite(IN3, LOW);
133     digitalWrite(IN4, LOW);
134 }

```

Figura 5: Continuación de la función “void loop()”

Después, dentro de la función “void loop” se usó el comando “if” para declarar variables:

- Como primera variable, cuando el botón 1 se presione, los motores avanzarán únicamente hacia adelante.
- Como segunda variable, cuando el botón 2 se presione, los motores irán únicamente hacia atrás.
- Como tercera variable, cuando el botón 3 se presione, se ejecutará la función en la cual los motores girarán hacia la derecha.
- Como cuarta variable, cuando el botón 4 se presione, se ejecutará la función en la que los motores girarán hacia la izquierda.
- Como última variable, cuando ningún botón esté presionado, se ejecutará la función “detener” en la cual ningún botón se moverá.

Por último, para establecer las funciones, se usaron 2 comandos: “digitalWrite” para establecer la dirección y “analogWrite” para darle velocidad a las ruedas. La función “adelante()” usa “digitalWrite” para enviar corriente a los dos lados en el mismo sentido (IN1 e IN3 en HIGH; IN2 e IN4 en LOW) y ajusta la velocidad deseada con “analogWrite” en ENA y ENB. La función “atras()” invierte las señales con “digitalWrite” (IN1 e IN3 en LOW; IN2 e IN4 en HIGH) para que todas las ruedas giren al revés.

La función “derecha()” hace un truco de giro: con “digitalWrite” manda las ruedas izquierdas hacia adelante (IN1=HIGH, IN2=LOW) y las derechas hacia atrás (IN3=LOW, IN4=HIGH) para que el carrito dé la vuelta sobre su propio centro. La función “izquierda()” es lo opuesto al giro anterior. Con “digitalWrite” manda las ruedas izquierdas hacia atrás (IN1=LOW, IN2=HIGH) y las derechas hacia adelante (IN3=HIGH, IN4=LOW). Finalmente, la función “detener()” pone la velocidad en cero usando “analogWrite”(ENA, 0) y “analogWrite” (ENB, 0), y apaga todos los pines con “digitalWrite” en LOW para cortar la corriente por completo y frenar de golpe.

III. Resultados

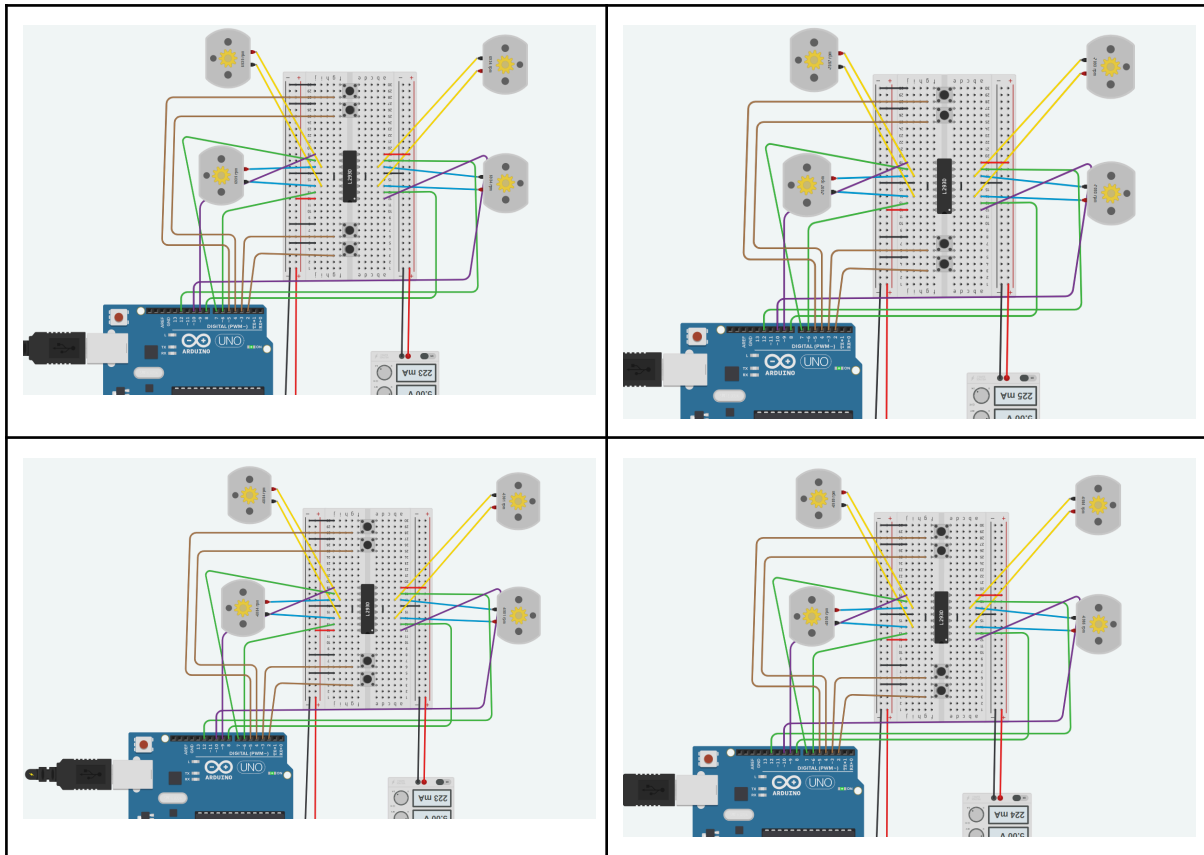


Figura 6: Circuito final en TinkerCAD

Como resultado final se obtuvo que, cuando se presiona el botón para avanzar hacia adelante, los motores giren hacia adelante, en otro botón que todos giren en sentido antihorario por lo que iría de reversa, en otro botón dos motores van en sentido horario y los otros dos en sentido antihorario, por lo que estos puedan girar.

IV. Referencias

- Gemini. (2026). *Gemini* (versión del 27 de septiembre) [Modelo de lenguaje grande]. <https://gemini.google.com>
- Prompt utilizado: “quiero que me expliques, de forma sencilla, esta codificación para un circuito en el cual se tienen 4 motores y se quiere que, a

través de botones, los motores avancen hacia adelante, hacia atrás, que giren hacia la izquierda o hacia la derecha”.